NXL-DS Hanwha

NXL-DS Hanwha 0.9 개발자 가이드

0.9.0 기준 · 2026-11-02

빠른 시작

처음이라면 이 순서대로 한다. 각 단계를 누르면 해당 절로 이동한다.

1	**패키지 설치** `pnpm add ./vendor/nextlab-ds-0.9.0.tgz`	2.1
2	**CSS 연결** `@import '@nextlab/ds/hanwha/styles.css';`	2.2
3	**폰트 파일 넣기** `public/fonts/` 에 한화고딕 3개	2.3
4	**Provider 로 감싸기** `<HanwhaProvider syncRoot linkComponent={Link}>`	2.4
5	**화면 만들기** 레이아웃은 5장, 컴포넌트는 6.4 선택 기준에서 고른다	6.4

1 개요

NXL-DS Hanwha 는 한화생명 고객 서비스 화면용 React 컴포넌트 라이브러리다. 디자이너의 Figma Component Library(NXL-DS)를 정본으로 삼아 컴포넌트·토큰·아이콘·그래픽을 제공하고, 화면 폭에 따른 전환(Mobile·Tablet·PC)을 컴포넌트와 토큰 수준에서 처리한다. 이 문서는 0.9.0 기준이다.

문서의 수치(파일 크기, 번들 크기, 색 대비, 시험 개수)는 0.9.0 배포물을 직접 재서 적었고, 측정 방법을 표 아래에 함께 둔다. 예시 코드는 모두 0.9.0 타입 선언에 strict 모드로 컴파일해 확인했다. 구조를 이렇게 정한 이유와 그 비용은 12장에 모았다.

1.1 구성

구분	내용
컴포넌트	입력·선택·데이터 표시·탐색·오버레이·로딩 80여 종 (하위 컴포넌트를 포함해 151개 export)
업무 컴포넌트	약관 동의 `TermsAgreement`, 서명 `SignaturePad`, 주소 검색 `AddressSearch`, 기관 선택 `InstitutionPicker`, 등급 표시 `LevelMeter`
레이아웃	Mobile: `Header` `BottomNavigation` `ButtonGroup fixed` / PC: `Header type="menu"` `PageContainer` `PageTitle` `PageColumns` `SideNavigation` `SiteFooter`
차트	`BarChart` `LineChart` `AreaChart` `DonutChart` (ApexCharts 기반)
아이콘	UI 아이콘 54종 (헤더용 27, 콘텐츠용 29. `close` `search` 는 양쪽에 있다)
그래픽	일러스트·피드백·아바타·로고·증권사 마크·인증 마크 213개, AI 생성 본문·배너 이미지 900개
디자인 토큰	색·타이포그래피·간격·라운드·그림자·모션. CSS 변수 504개(`--cds-*`), Tailwind 유틸리티, TS 상수(`CDS_*`)로 제공

1.2 진입점

import 경로	내용
`@nextlab/ds/hanwha`	컴포넌트, `HanwhaProvider`, 토큰 상수, `Icon`, `Graphic`
`@nextlab/ds/hanwha/icons`	`Icon` 과 아이콘 이름 목록
`@nextlab/ds/hanwha/graphics`	`Graphic` 과 그래픽 이름 목록
`@nextlab/ds/hanwha/styles.css`	컴포넌트 스타일, 토큰, 유틸리티를 미리 빌드한 CSS. 전송 크기는 brotli 38KB · gzip 70KB (10.1)

1.3 요구 사항

항목	조건
React, React DOM	19 이상 (peerDependencies)
TypeScript	`moduleResolution` 이 `bundler` · `node16` · `nodenext` 중 하나. `node` 는 하위 경로 exports 를 읽지 못해 `@nextlab/ds/hanwha` 의 타입을 찾지 못한다
모듈 형식	ESM 전용. CommonJS `require()` 로는 불러올 수 없다
Next.js	App Router. 16.4 에서 검증. 상태·이벤트가 있는 화면은 Client Component 로 작성 (2.5)
Vite	React 플러그인 프로젝트
Tailwind CSS	v4 권장. 화면 배치용 유틸리티에 사용 (3.4)
브라우저	iOS Safari 16.4, Chrome 111, Android Chrome 111, Edge 111 이상. CSS 가 `@property` · `@container` · `:has()` 를 쓴다
패키지 매니저	pnpm, npm, yarn 2 이상. yarn 1 은 tgz 설치가 실패한다

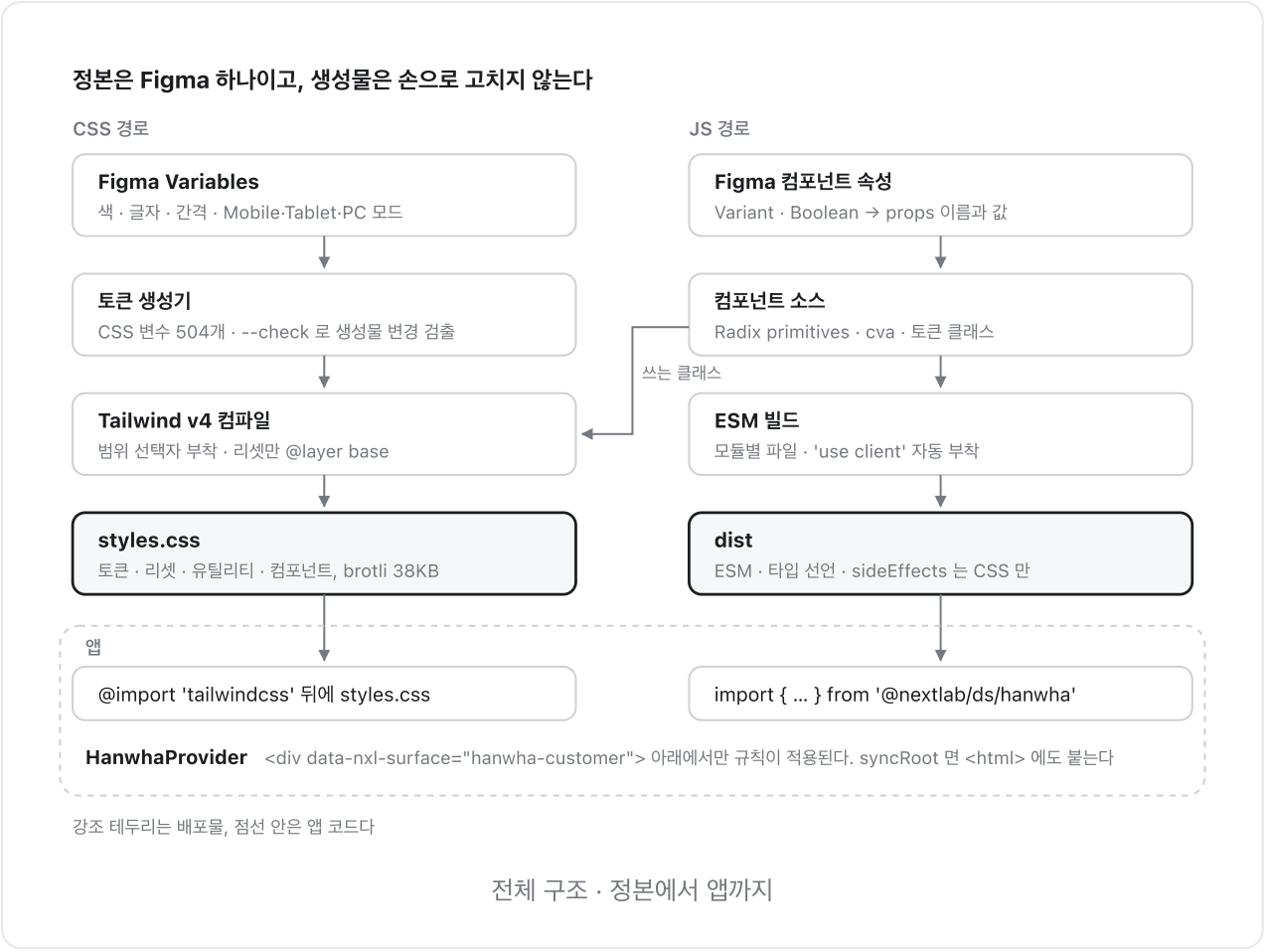
Radix UI primitives, class-variance-authority, tailwind-merge, clsx, ApexCharts 는 패키지 의존성으로 함께 설치된다.

1.4 참고 자료

자료	주소
Storybook (Chromatic)	https://main--6a794efb546779fb68f18bd0.chromatic.com
Storybook MCP	`https://main--6a794efb546779fb68f18bd0.chromatic.com/mcp`
한화 폰트 CDN	`https://nxl-nlc.hanwhalife.com/cnts-files/fonts/`

1.5 전체 구조

토큰과 컴포넌트는 Figma 에서 출발해 두 경로로 앱에 닿는다. CSS 경로는 Figma Variables 를 CSS 변수로 생성하고 Tailwind 로 한 번 컴파일해 `styles.css` 한 파일로 만든다. JS 경로는 Figma 컴포넌트 속성을 props 로 옮긴 소스를 모듈별 ESM 으로 빌드한다. 생성된 파일은 손으로 고치지 않으며, 값이 바뀌면 생성기를 다시 돌린다.



전체 구조 · 정본에서 앱까지

2 설치와 초기 설정

패키지 설치부터 첫 화면까지의 설정이다. 2.1~2.4 를 순서대로 하면 DS 컴포넌트가 화면에 나온다. Next.js 는 2.5, 테스트는 2.6 을 함께 본다.

2.1 패키지 설치

0.9.0 은 tgz 로 배포한다. 사내 Nexus 등록 후에는 레지스트리 설치로 바뀌지만 import 경로는 같다.

Shell

```
# tgz 배포본 (npm install, yarn add 도 같은 형식)
pnpm add ./vendor/nextlab-ds-0.9.0.tgz

# 사내 Nexus 등록 후
pnpm add @nextlab/ds

# Tailwind CSS v4
pnpm add -D tailwindcss @tailwindcss/postcss   # Next.js
pnpm add -D tailwindcss @tailwindcss/vite      # Vite
```

tgz 파일과 잠금 파일을 함께 커밋해 설치 결과를 고정한다. Tailwind 설정 파일은 Tailwind 공식 설치 문서를 따른다.

2.2 CSS

CSS

```
/* src/app/globals.css (Vite: src/index.css) */
@import 'tailwindcss';
@import '@nextlab/ds/hanwha/styles.css';
```

`styles.css` 는 컴포넌트 스타일, 디자인 토큰, 토큰 유틸리티를 한 파일로 미리 빌드한 것이다. 앱의 Tailwind 설정에 DS 토큰을 등록하지 않아도 `bg-bg-layer-base`, `type-body-medium` 같은 클래스가 동작한다. 구조는 3장에서 설명한다.

2.3 폰트

DS CSS 는 HanwhaGothic(300·400·700)을 `/fonts/HanwhaGothic{L,R,B}.woff2` 경로로 선언한다. 한화 폰트 CDN 의 파일을 이 경로에 둔다.

Shell

```
mkdir -p public/fonts
CDN=https://nxl-nlc.hanwhalife.com/cnts-files/fonts/HanwhaGothic
curl -o public/fonts/HanwhaGothicL.woff2 $CDN/06HanwhaGothicL.woff2   #
300
curl -o public/fonts/HanwhaGothicR.woff2 $CDN/05HanwhaGothicR.woff2   #
400
curl -o public/fonts/HanwhaGothicB.woff2 $CDN/04HanwhaGothicB.woff2   #
700
```

CDN 의 `index.css` 를 `<link>` 로 직접 참조하면 폰트 요청이 CORS 정책에 막힌다. CDN 이 `Access-Control-Allow-Origin` 헤더를 보내지 않기 때문이다.

파일은 굵기마다 약 1.2~1.3MB 다. `font-display: swap` 으로 선언되어 있어 폰트를 받는 동안에는 시스템 글꼴로 먼저 그리고, 파일이 없으면 font-family 순서에 따라 Pretendard, 시스템 글꼴로 대체된다. 본문 (`type-body-*`)은 300, 제목은 700 을 쓴다.

2.4 Provider

TSX

```tsx
// src/app/providers.tsx (Next.js)
'use client';

import Link from 'next/link';
import type { ReactElement, ReactNode } from 'react';
import { HanwhaProvider, ToastProvider, TooltipProvider } from
'@nextlab/ds/hanwha';

export function Providers({ children }: { children: ReactNode }):
ReactElement {
  return (
    <HanwhaProvider syncRoot linkComponent={Link}>
      <TooltipProvider>
        <ToastProvider>{children}</ToastProvider>
      </TooltipProvider>
    </HanwhaProvider>
  );
}
```

TSX

```tsx
// src/app/layout.tsx
import type { Viewport } from 'next';
import type { ReactElement, ReactNode } from 'react';
import './globals.css';
import { Providers } from './providers';

export const viewport: Viewport = {
  width: 'device-width',
  initialScale: 1,
  viewportFit: 'cover',
};

export default function RootLayout({ children }: { children: ReactNode
}): ReactElement {
  return (
    <html lang="ko">
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

`viewportFit: 'cover'` 가 있어야 iOS 에서 `env(safe-area-inset-bottom)` 값이 잡힌다. `ButtonGroup fixed`, `BottomNavigation`, `BottomSheet` 는 이 값만큼 홈 인디케이터 위로 띄워 그린다. Vite 는 `index.html` 에 `<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">` 를 둔다.

Vite 는 React Router 의 `Link` 를 `href` 를 받는 컴포넌트로 감싸 넘긴다.

TSX

```tsx
// src/providers.tsx (Vite + React Router)
import type { ReactElement, ReactNode } from 'react';
import { Link } from 'react-router';
import { HanwhaProvider, ToastProvider, TooltipProvider, type
HanwhaProviderProps } from '@nextlab/ds/hanwha';

const RouterLink: NonNullable<HanwhaProviderProps['linkComponent']> = ({
href, ...rest }) => <Link to={href} {...rest} />;

export function Providers({ children }: { children: ReactNode }):
ReactElement {
  return (
    <HanwhaProvider syncRoot linkComponent={RouterLink}>
      <TooltipProvider>
        <ToastProvider>{children}</ToastProvider>
      </TooltipProvider>
    </HanwhaProvider>
  );
}
```

TSX

```tsx
// src/main.tsx (Vite)
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import { App } from './app';
import { Providers } from './providers';

const container = document.getElementById('root');
if (container === null) throw new Error('#root 요소가 없다');

createRoot(container).render(
  <StrictMode>
    <Providers>
      <App />
    </Providers>
  </StrictMode>,
);
```

HanwhaProvider props

prop	값	기본값	설명
syncRoot	boolean	false	Provider 의 data 속성을 <html> 에도 적용한다. Portal 로 렌더링되는 오버레이에 필요하다. 켜면 DS 리셋이 문서 전체에 적용된다 (3.3)
linkComponent	컴포넌트	<a>	Header 메뉴, SiteMenu, SideNavigation, PageTitle 경로, SiteFooter 링크를 그릴 컴포넌트. 라우터의 Link 를 넘기면 페이지를 다시 받지 않고 이동한다
textScale	"standard" · "large"	"standard"	large 는 type-* 글자 크기와 줄 높이를 1.125배로 키운다
colorMode	"light" · "dark"	"light"	data-theme 으로 색 토큰 모드를 바꾼다. 다크 토큰은 검토 중 (14장)

prop	값	기본값	설명
audience	`"customer"` · `"sales"`	`"customer"`	화면 대상. 하위에서 `useHanwhaAudience()` 로 읽는다

`TooltipProvider` 는 `Tooltip` 을, `ToastProvider` 는 `useToast()` 를 쓸 때 필수다. 없으면 런타임 오류가 난다.

2.5 Next.js App Router

0.9.0 빌드는 React 훅, Context, 이벤트 핸들러, Radix 를 쓰는 모듈에 `'use client'` 를 자동으로 붙인다. 그래서 Server Component 에서도 DS 컴포넌트를 import 해 렌더링할 수 있다(Next.js 16.4 에서 확인). `Graphic` , `Feedback` , `Amount` , `Avatar` 처럼 상태가 없는 모듈은 지시어가 없어 서버에서만 그려진다. 다만 `'use client'` 모듈은 import 하는 모듈 전체를 클라이언트 번들로 가져가므로, `Header` 나 `List` 를 쓰는 앱에서는 그래픽 묶음도 클라이언트 번들에 들어간다 (10.3).

Next.js 는 Server Component 가 Client Component 에 함수를 props 로 넘기는 것을 허용하지 않는다. 작성 위치는 아래 기준으로 나눈다.

작성 위치	코드
Server Component	데이터 조회, 정적 표시(`Card` `List` `Amount` `Badge` `Tag` `Feedback` `Graphic` 등), `href` 로 이동하는 링크 (`<Button asChild><Link href="…">`)
Client Component (`'use client'`)	`onClick` · `onValueChange` 같은 이벤트 핸들러, 상태, 폼, `useToast()`, 오버레이 열기·닫기 제어

`buttonVariants` 처럼 `Variants` 로 끝나는 export 는 클라이언트 모듈에 있어 Server Component 에서 호출할 수 없다. 차트(`BarChart` 등)는 그리기 엔진을 브라우저에서 동적으로 불러오므로, 서버 HTML 에는 같은 크기의 빈 자리만 그려진다.

TSX

```tsx
// src/app/transfer/page.tsx (Server Component)
import type { ReactElement } from 'react';
import { Amount } from '@nextlab/ds/hanwha';
import { getAccounts, getBalance } from '@/features/transfer/api';
import { TransferForm } from './transfer-form';

export default async function TransferPage(): Promise<ReactElement> {
  const [accounts, balance] = await Promise.all([getAccounts(),
getBalance()]);
  return (
    <main className="flex flex-col gap-6 px-layout-margin pt-8 pb-12">
      <Amount value={balance} size="large" prefix="출금 가능" />
      <TransferForm accounts={accounts} />
    </main>
  );
}
```

TSX

```tsx
// src/app/transfer/transfer-form.tsx (Client Component)
'use client';

import { useState, type ReactElement } from 'react';
import { Button, ButtonGroup, SelectField, TextField } from
'@nextlab/ds/hanwha';
import type { Account } from '@/features/transfer/types';

interface TransferFormProps {
  accounts: readonly Account[];
}

export function TransferForm({ accounts }: TransferFormProps):
ReactElement {
  const [accountId, setAccountId] = useState<string>();

  return (
    <>
      <SelectField
        label="출금 계좌"
        options={accounts.map((a) => ({ value: a.id, label: a.name }))}
        value={accountId}
        onValueChange={setAccountId}
      />
      <TextField label="보낼 금액" format="amount" suffix="원" />
      <ButtonGroup fixed primary={<Button disabled={accountId ===
undefined}>다음</Button>} />
    </>
  );
}
```

2.6 테스트 환경

컴포넌트 테스트는 Vitest, Testing Library, jsdom 을 기준으로 한다. 패키지가 ESM 전용이라 Jest 는 별도 변환 설정이 필요하다.

jsdom 에 없는 브라우저 API 는 setup 파일에서 채운다. 오버레이·탭·목록은 `ResizeObserver` 와 `scrollIntoView` 를, 서명 패드는 pointer capture 를, 차트는 `matchMedia` 를 쓴다.

TS

```ts
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: { environment: 'jsdom', setupFiles: ['./vitest.setup.ts'] },
});
```

TS

```ts
// vitest.setup.ts
import '@testing-library/jest-dom/vitest';

class ResizeObserverStub implements ResizeObserver {
  observe(): void {}
  unobserve(): void {}
  disconnect(): void {}
}
globalThis.ResizeObserver ??= ResizeObserverStub;

Element.prototype.scrollIntoView ??= function scrollIntoView(): void {};
Element.prototype.hasPointerCapture ??= () => false;
Element.prototype.setPointerCapture ??= () => {};
Element.prototype.releasePointerCapture ??= () => {};

window.matchMedia ??= (query: string): MediaQueryList => ({
  matches: false,
  media: query,
  onchange: null,
  addListener: () => {},
  removeListener: () => {},
  addEventListener: () => {},
  removeEventListener: () => {},
  dispatchEvent: () => false,
});
```

TSX

```tsx
// src/test/render.tsx
import { render, type RenderOptions, type RenderResult } from '@testing-library/react';
import type { ReactElement, ReactNode } from 'react';
import { HanwhaProvider, ToastProvider, TooltipProvider } from '@nextlab/ds/hanwha';

function Wrapper({ children }: { children: ReactNode }): ReactElement {
  return (
    <HanwhaProvider>
      <TooltipProvider>
        <ToastProvider>{children}</ToastProvider>
      </TooltipProvider>
    </HanwhaProvider>
  );
}

export function renderWithDs(ui: ReactElement, options?: Omit<RenderOptions, 'wrapper'>): RenderResult {
  return render(ui, { wrapper: Wrapper, ...options });
}
```

동작·접근성 테스트에는 CSS 를 불러오지 않아도 된다. 요소는 역할과 이름으로 찾는다 (`screen.getByRole('button', { name: '다음' })`). 다이얼로그·바텀시트 내용은 `document.body` 에 그려지므로 `screen` 으로 찾는다.

---

3 스타일 아키텍처

`styles.css` 는 Tailwind v4 로 미리 컴파일한 CSS 를 하나의 범위 선택자 아래에 가둔 파일이다. 앱 CSS 와 충돌하지 않는 이유, 그리고 앱에서 지켜야 하는 조건이 이 구조에서 나온다.

3.1 범위 선택자

모든 규칙은 `[data-nxl-surface="hanwha-customer"]` 아래에서만 적용된다. 이 속성은 `HanwhaProvider` 가 렌더링하는 래퍼 `<div>` 에 붙는다. Provider 바깥 요소에는 DS 의 리셋·토큰·유틸리티가 적용되지 않는다.

HTML

```
<div data-nxl-surface="hanwha-customer" data-nxl-audience="customer"
data-theme="light" data-nxl-text-scale="standard">
  <!-- 앱 화면 -->
</div>
```

속성	값	역할
data-nxl-surface	hanwha-customer	CSS 범위
data-theme	light · dark	색 토큰 모드
data-nxl-text-scale	standard · large	글자 확대 배율 --cds-text-zoom (1 · 1.125)
data-cds-scale	mobile · tablet · pc	타이포·간격 스케일 강제. 지정하지 않으면 미디어 쿼리(768·1200px)로 결정된다

3.2 Cascade layer 와 우선순위

DS CSS 에서 레이어 안에 있는 것은 리셋(Tailwind preflight) 하나다. 토큰, 유틸리티, 컴포넌트 규칙은 모두 레이어 밖에 있다. CSS 는 레이어 밖 규칙을 레이어 안 규칙보다 명시도와 무관하게 우선한다. 앱 CSS 와 만났을 때의 결과는 아래와 같다.

DS 규칙	위치	앱 CSS 와 만나면
리셋 (preflight)	`@layer base`	앱 Tailwind 유틸리티 (`@layer utilitie` 가 이긴다. 앱이 준 `p-` 가 리셋의 `padding:` 에 덮이지 않는다
토큰·유틸리티·컴포넌트	레이어 밖. `[data-nxl-surface="hanwha-customer"] .flex` 처럼 범위 선택자가 붙어 명시도가 (0,2,0) 이상이다	앱 Tailwind 유틸리티 레이어 안이라 이 규칙 이기지 못한다. 앱의 레이어 밖 일반 CSS 는 명시도로 겨루며, `.card { … }`(0,1,0 는 진다

컴포넌트의 `className` 이 동작하는 것은 `cn()` (clsx + tailwind-merge)이 같은 속성의 내부 클래스를 빼 주기 때문이다. `h-14` 를 넘기면 내부의 `h-(--cds-dimension-x12)` 가 빠지고 `h-14` 만 남는다. 상태 변형(`data-[state=error]:…`)이나 하위 요소 선택자(`[&_svg]:…`)로 걸린 내부 규칙은 빠지지 않고, 레이어 밖이라 넘긴 클래스보다 우선한다.

그래서 `className` 은 배치(margin, width, flex·grid 항목 속성)에만 쓴다. 높이·색·내부 간격을 `className` 으로 바꾸면 일부 상태에서만 적용되고, 토큰이 바뀌어도 그 화면에는 반영되지 않는다.

3.3 Portal 과 syncRoot

`Dialog` , `BottomSheet` , `Tooltip` , `Menu` , `SelectField` 펼침 목록 같은 오버레이는 Radix Portal 로 `document.body` 아래에 렌더링된다. 이 위치는 Provider 래퍼 밖이라 범위 선택자에 걸리지 않는다. `syncRoot` 를 켜면 Provider 가 `<html>` 에 같은 data 속성을 설정해 Portal 내용에도 토큰과 스타일이 적용된다. 앱 루트의 Provider 에는 항상 켠다.

`syncRoot` 는 `useLayoutEffect` 에서 `<html>` 의 속성을 바꾸고, Provider 가 사라지면 이전 값으로 되돌린다. 서버 HTML 에는 래퍼 `<div>` 의 속성이 이미 들어 있으므로 첫 화면은 속성 적용 전에도 스타일이 맞다. `<html>` 이 범위가 되므로 그 뒤로는 Provider 바깥 요소에도 DS 리셋과 3.6 의 규칙이 적용된다.

3.4 미리 컴파일된 유틸리티와 앱 Tailwind

`styles.css` 에는 아래 클래스가 미리 들어 있다.

종류	예
색 토큰	`text-fg-*` `bg-bg-*` `border-stroke-*`
타이포그래피	`type-*`
형태	`rounded-*` `shadow-elevation-*`
레이아웃 간격 토큰	`px-layout-margin` `gap-layout-gutter` 등 (반응형 변형 포함)
DS 소스가 쓰는 Tailwind 기본 유틸리티	`flex` `items-center` `w-full` 등

DS 가 미리 만들지 않은 클래스(예: `grid-cols-3`)는 앱 Tailwind 가 생성한다. 앱 Tailwind 는 DS 토큰 이름을 모르므로 토큰 클래스는 DS CSS 가, 그 외 배치 클래스는 앱 Tailwind 가 맡는다. `gap-(--cds-size-base-md)` 처럼 CSS 변수를 직접 참조하는 클래스는 앱 Tailwind 가 생성하고, 값은 DS 범위의 CSS 변수에서 온다.

3.5 토큰 클래스와 변형 접두사

앱 Tailwind 는 DS 토큰 이름을 모르므로, 토큰 클래스에 변형 접두사를 붙인 조합은 앱에서 생성되지 않는다. DS CSS 에는 DS 소스가 직접 쓰는 조합과 간격 토큰의 `md:` · `lg:` 변형만 들어 있다. 그 밖의 조합은 CSS 변수를 참조하는 문법으로 쓴다.

의도	생성되지 않는 클래스	대신 쓰는 클래스
hover 시 회색 면	`hover:bg-bg-neutral-tint`	`hover:bg-(--cds-bg-neutra`
hover 시 브랜드 글자	`hover:text-fg-brand-strong`	`hover:text-(--cds-fg-brar`
포커스 시 테두리	`focus:border-stroke-focus-default`	`focus:border-(--cds-strok`

의도	생성되지 않는 클래스	대신 쓰는 클래스
부모 hover 시 글자색	`group-hover:text-fg-brand-default`	`group-hover:text-(--cds-f`

변수 이름은 클래스에서 CSS 속성 접두사를 떼고 `--cds-` 를 붙인 것이다(`bg-bg-neutral-tint` → `--cds-bg-neutral-tint` , `px-layout-margin` → `--cds-layout-margin`). `type-*` 는 여러 속성을 묶은 클래스라 변형 접두사와 함께 쓰지 않는다. 글자 크기는 분기점에서 이미 자동으로 바뀐다. 누름·hover 덮개는 색을 직접 칠하지 않고 `state-layer interaction-overlay` 를 쓴다(6.8).

3.6 앱 변형 클래스가 무시되는 경우

3.2 의 결과로, DS CSS 에 들어 있는 클래스와 같은 속성을 바꾸는 앱의 변형 클래스를 한 요소에 함께 쓰면 Provider 안에서는 앱 쪽이 무시된다. 오류나 경고 없이 화면만 다르게 나온다. 아래는 Chromium 에서 계산된 `display` 값이다.

클래스	화면 폭	Provider 밖	Provider 안
`flex min-[768px]:hidden`	800px	`none`	`flex` (숨겨지지 않음)
`block min-[768px]:flex`	800px	`flex`	`block`
`flex lg:hidden`	1100px	`none`	`flex`
`max-[767px]:flex min-[768px]:hidden`	400 · 800px	`flex` · `none`	`flex` · `none`

`flex` `block` `hidden` 은 DS 소스가 쓰므로 DS CSS 에 레이어 밖 규칙으로 들어 있다. `min-[768px]:hidden` 은 DS CSS 에 없어 앱 Tailwind 가 레이어 안에 만들고, 레이어 밖의 `flex` 에 진다. `flex min-[1200px]:hidden` 처럼 DS CSS 에 마침 변형까지 들어 있는 조합은 동작하지만, 어떤 클래스가 들어 있는지는 버전마다 달라진다.

Provider 안에서 화면 폭이나 상태에 따라 같은 속성을 바꿀 때는 조건 없는 기본값 클래스를 두지 않는다. 모든 구간을 범위 변형으로 쓰면(`max-[767px]:flex min-[768px]:hidden`) DS CSS 의 내용과 무관하게 의도대로 적용된다. 레이아웃 전환이 목적이면 이미 화면 폭에 따라 바뀌는 `PageColumns` `Header` 같은 컴포넌트를 먼저 쓴다.

4 디자인 토큰

화면 코드의 색·글자·간격·모서리는 이 장의 토큰 클래스로만 쓴다. 값을 직접 쓰면 디자인 변경과 큰 글자 모드가 반영되지 않는다.

4.1 계층

토큰은 디자이너의 Figma Variables 에서 생성된다. Variables 를 내보낸 JSON 을 생성기가 읽어 원색, 시맨틱(light·dark), 반응형 CSS 를 만든다. 반응형 토큰은 `:root` 기본값(Mobile)에 768px · 1200px 미디어 쿼리 값과 `[data-cds-scale]` 강제 값을 더한 형태다. 생성기의 `--check` 모드는 커밋된 CSS 가 생성 결과와 한 글자라도 다르면 실패하므로, 손으로 고친 값이 섞이지 않는다. 값 변경은 Figma Variables 에서 시작한다.

층	예	쓰임
Figma Variables	`fg/neutral/default`	디자인 정본
원색 팔레트	`--cds-orange-500` `--cds-gray-900`	토큰 정의에만 쓴다. 화면 코드에서 참조하지 않는다
시맨틱 토큰	`--cds-fg-neutral-default`	`data-theme` 에 따라 light·dark 값이 바뀐다
유틸리티 클래스	`text-fg-neutral-default`	화면 코드가 쓰는 층
TS 상수	`CDS_TYPOGRAPHY` `CDS_LAYOUT` `CDS_BREAKPOINTS`	JS 에서 값이 필요할 때 (차트, 캔버스 등)

4.2 색

클래스 이름은 `{CSS 속성}-{그룹}-{톤}-{세기}` 순서다. 그룹은 `fg` (글자·아이콘), `bg` (면), `stroke` (선) 이고, 톤은 `neutral` `brand` `critical` `warning` `positive` `info` `disabled` 다. `fg` 토큰은 `text-` `fill-` `stroke-`, `stroke` 토큰은 `border-` `outline-` `ring-` `divide-` 접두사로 쓸 수 있다.

`bg-bg-layer-base` 의 겹친 `bg` 는 Tailwind 속성 접두사 뒤에 Figma 변수 경로(`bg/layer/base`)를 그대로 붙여서 생긴다. 줄여 쓰지 않은 대신 클래스만 보고 Figma 변수를 찾을 수 있고, 디자인 명세의 변수 이름을 그대로 옮기면 클래스가 된다.

용도	클래스
페이지 배경	`bg-bg-layer-base`
카드·패널	`bg-bg-layer-default`
바텀시트·팝오버	`bg-bg-layer-floating`
회색 구역	`bg-bg-neutral-tint` · `bg-bg-neutral-tint-strong`
브랜드 면	`bg-bg-brand-solid` · `bg-bg-brand-tint`
반전 면(토스트·툴팁)	`bg-bg-inverse-default` + `text-fg-inverse-default`
제목·본문	`text-fg-neutral-default` · `text-fg-neutral-strong`
보조 텍스트	`text-fg-neutral-muted` → `subtle` → `tertiary` (뒤로 갈수록 연함. 본문 크기는 `muted`, 9.3)
브랜드 텍스트	`text-fg-brand-default` · 본문 크기는 `text-fg-brand-strong` (9.3)
상태 텍스트	`text-fg-critical-default` · `warning` · `positive` · `info`
금액 등락	`text-fg-trend-up`(상승, 빨강) · `text-fg-trend-down`(하락, 파랑) · `text-fg-trend-flat`
진한 면 위 텍스트	`text-fg-neutral-inverse`
비활성	`text-fg-disabled-default` · `bg-bg-disabled-default`
구분선·테두리	`border-stroke-neutral-subtle` · `default` · `strong`
포커스	`outline-stroke-focus-default`

hex 값과 원색 팔레트 클래스(`bg-orange-500`)는 화면 코드에 쓰지 않는다. 다크 모드와 토큰 값 변경이 반영되지 않는다.

4.3 타이포그래피

`type-{역할}-{크기}` 클래스 하나가 font-size, line-height, letter-spacing, font-weight 를 정한다. 색은 `text-fg-*` 로 따로 지정한다. 크기는 화면 폭에 따라 아래 값으로 바뀌고, `textScale="large"` 일 때 1.125배가 된다.

클래스	굵기	Mobile	Tablet	PC
type-display-large	700	48/60	48/60	56/70
type-display-medium	700	40/50	42/54	48/60
type-display-small	700	36/46	36/46	40/52
type-heading-large	700	32/40	32/40	36/46
type-heading-medium	700	28/36	28/36	32/42
type-heading-small	700	24/34	24/34	28/40
type-title-large	700	22/32	22/32	24/36
type-title-medium	700	20/28	20/28	22/32
type-title-small	700	18/26	18/26	20/30
type-body-large	300	16/24	16/24	18/28
type-body-medium	300	14/22	14/22	16/26
type-body-small	300	13/18	13/18	14/20
type-caption-medium	300	12/18	12/18	12/18
type-caption-small	300	11/18	11/18	11/18

값은 font-size/line-height(px)다. `title-*-subtle` 은 굵기 400, `body-*-strong` · `caption-*-strong` 은 굵기 700 변형이다. 밑줄 링크 글자는 클래스가 아니라 `Link` 컴포넌트로 쓴다. Tailwind 의 `text-sm`, `font-bold`, `leading-*` 는 `--cds-text-zoom` 과 반응형 전환을 따르지 않으므로 텍스트에 쓰지 않는다.

4.4 간격

스케일	정의	쓰는 법
dimension (고정)	`--cds-dimension-x{n}` = n × 4px (x1 … x18)	Tailwind 숫자 유틸리티와 같은 값 (p-4 = 16px = x4)
size-base (반응형)	`--cds-size-base-{xs,sm,md,lg,xl,xxl}`	화면 폭에 따라 값이 바뀜. `gap-(--cds-size-base-m`
화면 여백·거터	`layout-margin` `layout-gutter` (16 · 24 · 32px)	`px-layout-margin` `gap-layout-gutter`
섹션 간격	`layout-section-gap` (24 · 32 · 40px)	`gap-layout-section-gap`
카드 안쪽 여백	`layout-card-padding` (16 · 20 · 24px)	`p-layout-card-padding`

화면 여백부터 카드 여백까지 괄호 안 값은 Mobile · Tablet · PC 순서다.

4.5 라운드·그림자

토큰	값
`rounded-r1` · `r2` · `r3` · `r4` · `r5` · `r6`	4 · 8 · 12 · 16 · 20 · 24px (중간값 `r0-5` 2, `r1-5` 6, `r2-5` 10, `r3-5` 14)
`rounded-full`	알약형
`shadow-elevation-small` · `medium` · `large`	카드 · 세그먼트 · 다이얼로그·바텀시트

4.6 모션

토큰	값	쓰는 법
`--cds-duration-short` · `base` · `long`	120 · 200 · 300ms	`duration-(--cds-dura`
`--cds-easing-standard`	`cubic-bezier(0.2, 0, 0, 1)`	`ease-(--cds-easing-s`
`--cds-easing-decelerate`	`cubic-bezier(0, 0, 0, 1)`	등장 애니메이션. `ease-(--cds-easing-d`

컴포넌트 애니메이션은 `prefers-reduced-motion` 에서 멈추거나 줄어든다. 화면에서 직접 넣는 애니메이션도 `motion-safe:` 로 감싼다.

4.7 쌓임 순서

클래스	z-index	쓰는 곳
`z-sticky`	10	고정 머리(`Header sticky`), 하단 고정 버튼
`z-overlay`	100	다이얼로그, 바텀시트, 메뉴, 펼침 목록, 툴팁
`z-toast`	200	토스트

앱에서 직접 만드는 고정 요소는 `z-sticky` 를 쓰거나 100 보다 낮게 둔다. 100 이상을 쓰면 DS 오버레이 위로 올라온다.

---

5 레이아웃과 반응형

화면은 768px 과 1200px 에서 바뀐다. 모바일은 `Header` 와 하단 고정 버튼, PC 는 메뉴형 `Header` 와 `PageContainer` 로 짠다.

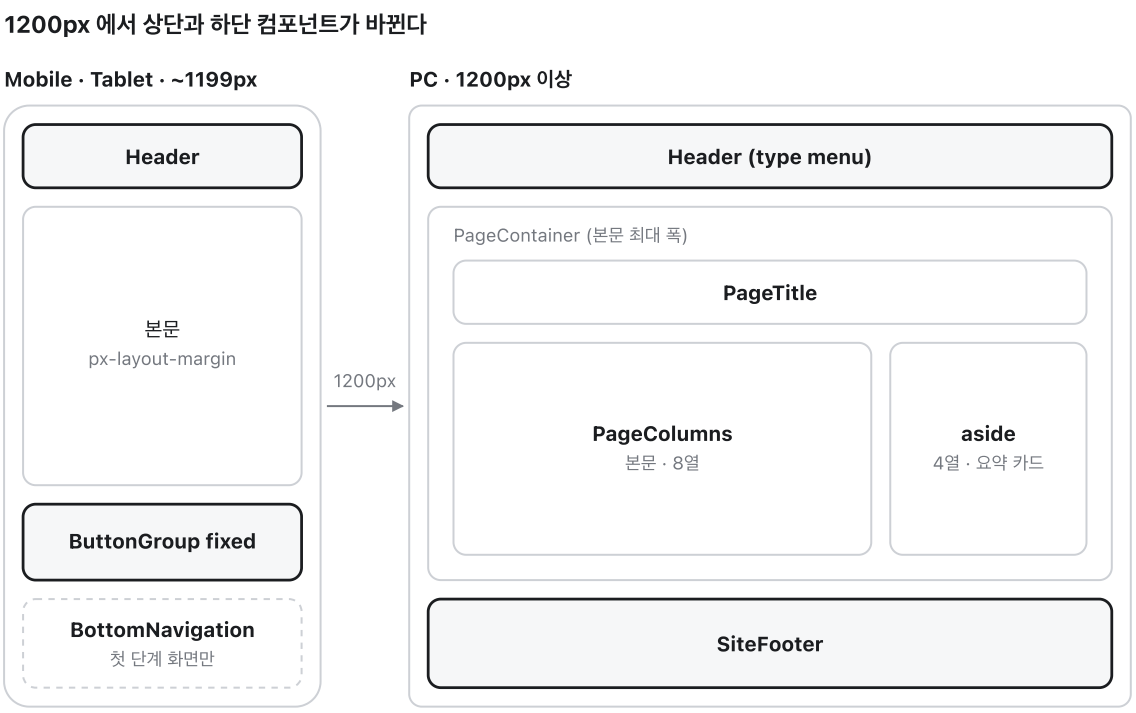
5.1 분기점과 그리드

구간	화면 폭	좌우 여백	거터	열	본문 최대 폭
Mobile	0–767px	16px	16px	4	제한 없음
Tablet	768–1199px	24px	24px	8	제한 없음
PC	1200px 이상	32px	32px	12 (density middle) · 8 (low)	1176px (middle) · 720px (low)

좌우 여백과 거터는 `px-layout-margin`, `gap-layout-gutter` 로 쓰며 값은 분기점에서 바뀐다. 타이포그래피와 `size-base` 간격도 같은 분기점에서 자동으로 바뀐다. 버튼·입력 필드 높이는 화면 폭과 무관하게 고정이다.

코드에서 분기할 때는 Tailwind 기본 `lg:` (1024px) · `xl:` (1280px) 대신 `min-[768px:]` · `min-[1200px:]` 을 쓴다. Provider 안에서는 기본값 클래스 없이 구간마다 범위 변형으로 쓴다(3.6). JS 에서는 `CDS_BREAKPOINTS` (`{ tablet: 768, desktop: 1200 }`)를 읽는다.

토큰은 화면 폭(미디어 쿼리)을 따르고, `PageColumns` 와 PC `Header` 는 자기 폭(컨테이너 쿼리)을 따른다. 그래서 PC 화면이라도 좁은 영역 안에 넣은 `PageColumns` 는 한 칸으로 그려진다.



강조 테두리는 1200px 에서 바뀌는 컴포넌트, 점선은 화면에 따라 넣는 컴포넌트다

화면 틀 · Mobile·Tablet 과 PC

5.2 Mobile·Tablet 구조

TSX

```tsx
<div className="flex min-h-dvh flex-col bg-bg-layer-base">
  <Header title="내 보험" onLeftClick={goBack} />
  <main className="flex-1 px-layout-margin pt-8 pb-12">{/* 본문 */}</main>
  <ButtonGroup fixed primary={<Button>확인</Button>} />
</div>
```

영역	규칙
`Header`	기본 `layout="compact"`(56px). 왼쪽 뒤로가기, 오른쪽 아이콘 0–3개, 제목 한 줄(넘치면 말줄임). 제목은 기본 `h1` 이며 본문에 별도 페이지 제목이 있으면 `titleAs="p"`
`ButtonGroup fixed`	화면 하단 고정 버튼 영역. `primary` 하나, 또는 `secondary` + `primary` 두 개를 둔다

영역	규칙
`BottomNavigation`	항목 3–5개. 최상위(1depth) 화면에만 두고 PC 에서는 쓰지 않는다. 선택 항목은 BottomNavigationItem 의 `selectedIcon`(채운 아이콘)

5.3 PC 구조

TSX

```tsx
// src/app/product/calc/calc-screen.tsx
'use client';

import { useRouter } from 'next/navigation';
import type { ReactElement } from 'react';
import {
  Header,
  PageColumns,
  PageContainer,
  PageTitle,
  SiteFooter,
  type HeaderMenuItem,
} from '@nextlab/ds/hanwha';
import { PremiumSummary } from './premium-summary';

const MENU = [
  { id: 'product', label: '보험상품', href: '/product' },
  { id: 'claim', label: '보험금 청구', href: '/claim' },
  { id: 'my', label: '마이페이지', href: '/my' },
] satisfies HeaderMenuItem[];

export function CalcScreen(): ReactElement {
  const router = useRouter();
  return (
    <div className="flex min-h-dvh flex-col bg-bg-layer-base">
      <Header type="menu" items={MENU} activeId="product" compactTitle="보
험료 계산" onLeftClick={() => router.back()} />
      <PageContainer as="main" className="flex-1">
        <PageTitle title="보험료 계산" description="나이와 성별을 넣으면 월 보험료를
계산해 드려요." />
        <PageColumns aside={<PremiumSummary />}>{/* 입력·설명 */}
</PageColumns>
      </PageContainer>
      <SiteFooter links={[{ label: '개인정보처리방침', href: '/privacy',
emphasis: true }]} />
    </div>
```

```
  );
}
```

컴포넌트	동작	주요 props
`Header type="menu"`	1200px 이상에서 메뉴형 PC 헤더. 헤더 폭이 1200px 미만이면 모바일 헤더와 전체 메뉴(`SiteMenu`)로 바뀐다	`items` `activeId` `onNavigate` `actions` `compactTitle` `onLeftClick` `menu` `menuLayout`
`Header layout="wide"` · `"contained"`	가입·청구 같은 집중 과업 화면의 PC 헤더(64px). 뒤로가기, 과업 이름, 닫기를 둔다. 진행 단계(Stepper)는 헤더가 아니라 본문 첫머리에 둔다	`title` `onLeftClick` `rightActions`
`PageContainer`	좌우 여백과 본문 최대 폭	`density`(`middle` · `low`), `as`
`PageTitle`	본문 첫 제목과 머리-제목-본문 간격	`title` `description` `breadcrumb` `actions` `pcOnly`
`PageColumns`	본문 + 보조 패널(`aside`). 자기 폭 960px 이상에서 두 칸	`aside` `split`(`8:4` · `9:3` · `7:5`) `stickyAside` `mobileAside`(`stack` · `hidden`)
`SideNavigation`	마이페이지·고객센터 같은 구역의 왼쪽 메뉴	`title` `items` `activeId` `onNavigate`
`SiteFooter`	약관·정책 링크와 회사 정보	`links` `info` `copyright` `color`

`PageContainer`, `PageTitle`, `PageColumns`, `SiteFooter`, `SiteMenu` 는 Figma 정의 전의 확장 컴포넌트로 디자이너 검토 중이다. props 이름이 1.0.0 전에 바뀔 수 있다 (14장).

6 컴포넌트 API 규약

모든 컴포넌트가 따르는 props 규칙과 상황별 선택 기준이다. 어떤 컴포넌트를 쓸지 모르겠으면 6.4 의 표부터 본다.

6.1 구조

상호작용 컴포넌트는 Radix UI primitives(Dialog, Select, Tabs, Checkbox, RadioGroup, Switch, Accordion, Tooltip, Popover, Progress) 위에 구현했다. 키보드 조작, 포커스 관리, ARIA 속성은 Radix 의 동작을 따른다. 변형은 class-variance-authority 로 정의하고, 클래스 병합은 `cn()` (clsx + tailwind-merge)으로 한다.

6.2 props 규칙

항목	규칙
Figma 속성 대응	props 이름과 값은 Figma 컴포넌트 속성을 따른다. 각 prop 의 JSDoc 에 대응하는 Figma 속성이 적혀 있다 (예: `Button` 의 `variant` = Figma Type, `size` = Figma Size)
HTML 속성	루트 요소의 HTML 속성을 그대로 받는다. `TextField` 는 `<input>` 속성을 받는다
ref	`forwardRef` 로 실제 DOM 요소를 전달한다. `TextField` 의 ref 는 `HTMLInputElement` 다
`className`	루트 요소에 병합된다. 배치용으로만 쓴다 (3.2)
`asChild`	`Button`, `BottomNavigationItem` 은 Radix Slot 으로 자식 요소에 스타일과 동작을 넘긴다
영역 props	Figma 에서 영역 구성이 정해진 컴포넌트(`DialogContent` `BottomSheetContent` `FieldArea` `Feedback`)는 하위 컴포넌트 조합 대신 `title` `footer` `label` `errorText` 같은 영역 props 로 받는다. 영역의 순서·간격·낭독 연결은 컴포넌트가 정한다

항목	규칙
`status` 와 `critical`	결과·상황을 뜻하는 `status` 값은 `error` 를 쓰고(Feedback `useToast`), 색 토큰은 `critical` 을 쓴다(`text-fg-critical-default`). `Button color="critical"` 은 0.9.0 에서 사용 중단됐다(Figma 에 없음, 1.0.0 에서 제거)
제어·비제어	선택형 컴포넌트는 `value` · `defaultValue` · `onValueChange`(Radix 규칙), 입력형은 HTML 의 `value` · `onChange` 를 따른다
`state`	Storybook 용 정지 상태 표시다. 실제 상태는 `disabled` · `readOnly` · `invalid` · 포커스로 결정되므로 화면 코드에서 쓰지 않는다
`inline`	`Dialog` 등을 Portal 없이 문서 흐름에 그리는 문서용 옵션이다. 화면 코드에서 쓰지 않는다
`@deprecated` 표시	에디터 자동완성에서 취소선으로 보이는 이름은 이전 화면 호환용이다. 쓰지 않고, 툴팁 설명에 적힌 대체 이름을 쓴다

TSX

```tsx
// Next.js Link 를 버튼 모양으로
import Link from 'next/link';
import { Button } from '@nextlab/ds/hanwha';

<Button asChild>
  <Link href="/apply">가입하기</Link>
</Button>
```

6.3 컴포넌트 목록

분류	컴포넌트
버튼	`Button` `ButtonGroup` `TextButton` `IconButton` `IconLabelButton` `Link`
입력	`TextField` `PasswordField` `TextArea` `SearchField` `SelectField` `DateField` `DateRangeField` `FieldSet`

분류	컴포넌트
선택	`Checkbox` `Radio` `RadioGroup` `Switch` `Chip` `ChipGroup` `Segment` `SelectionCard` `SelectionCardGroup`
데이터 표시	`Card` `List` `ListItem` `ListAmount` `CardAmount` `Table` `DataTable` `Pagination` `Tabs` `Accordion` `AccordionGroup` `Badge` `Tag` `Avatar` `Divider` `Text` `TextBullet` `Amount` `AmountChange` `TermsAccordion` `Banner` `ImageBanner` `Carousel` `Panel`
진행·상태	`Stepper` `ProgressBar` `ProgressCircle` `PageIndicator` `Spinner` `Skeleton` `SkeletonGroup` `Loading` `Feedback` `Rating` `LevelMeter`
차트	`BarChart` `LineChart` `AreaChart` `DonutChart` `CompareBars` `RatioBar` `ChartLegend`
오버레이	`Dialog` `AlertDialog` `BottomSheet` `Menu` `Toast`(`useToast`) `Tooltip`
날짜	`DatePicker` `Calendar`
업무	`TermsAgreement` `TermsDesign` `TermsMarketing` `TermsSignature` `SignaturePad` `AddressSearch` `InstitutionPicker`
레이아웃	`Header` `BottomNavigation` `PageContainer` `PageTitle` `PageColumns` `SideNavigation` `SiteFooter` `SiteMenu`

6.4 선택 기준

요구	컴포넌트	기준
텍스트·숫자 입력	`TextField`	단위는 `suffix`, 금액은 `format="amount"`(쉼표와 한글 금액 표시), 지우기는 `clearable`
단일 선택 2–4개	`Segment`	라벨이 짧아 한 줄에 들어갈 때. 상한은 `SEGMENT_MAX_ITEMS`(4)
단일 선택	`RadioGroup`	라벨이 길거나 설명이 붙을 때

요구	컴포넌트	기준
카드형 선택	`SelectionCardGroup`	상품·플랜처럼 항목마다 설명이 있을 때
단일 선택, 항목이 많을 때	`SelectField`	목록이 화면 폭에 따라 자동으로 바뀐다. Mobile 은 아래 시트, Tablet·PC 는 가운데 창(Dialog). PC 에서 항목이 6개 이하면 칸 바로 아래 펼침 목록. 검색이 필요한 목록에는 쓰지 않는다
다중 선택	`Checkbox` · `ChipGroup`	
날짜·기간	`DateField` · `DateRangeField`	달력(`DatePicker`) 내장
부가 작업·긴 목록 선택	`BottomSheet`	태블릿(768px~)부터 Dialog 로 그린다. 끄려면 `BottomSheetContent` 에 `dialogFromTablet={false}`
확인·취소	`Dialog` · `AlertDialog`	되돌릴 수 없는 작업은 AlertDialog. 확인 버튼은 맨 오른쪽 (6.6)
더보기(⋯) 행동	`Menu`	
일시 알림	`useToast().show()`	기본 4초 후 닫힘. `action` 이 있으면 자동으로 닫히지 않음. `status` 로 상태 아이콘
하단 주 행동	`ButtonGroup fixed`	
단계 진행	`ProgressBar step` · `Stepper`	
목록·카드 로딩	`Skeleton` · `SkeletonGroup`	내용 자리를 미리 그릴 때
짧은 대기	`Spinner`	
화면 전체 처리 중	`Loading`	`type overlay` · `screen`

요구	컴포넌트	기준
빈 상태·오류·완료	`Feedback`	`type screen` · `area` × `status success` · `empty` · `error`
금액 표시	`Amount` · `AmountChange` · `ListAmount` · `CardAmount`	값이 없으면 0원이 아니라 "미확인"으로 표시

6.5 비동기 화면

데이터를 조회하는 화면은 로딩, 빈 상태, 오류를 모두 구현한다.

TSX

```tsx
const { data, isPending, isError, refetch } = useQuery(contractsQuery);

if (isPending) {
  return (
    <SkeletonGroup label="계약 목록을 불러오는 중" className="flex flex-col
gap-4">
      <Skeleton size="title" />
      <Skeleton size="body-medium" />
      <Skeleton size="body-medium" />
    </SkeletonGroup>
  );
}
if (isError) {
  return (
    <div className="flex flex-col items-center gap-2">
      <Feedback type="area" status="error" title="계약 목록을 불러오지 못했어요"
/>
      <TextButton color="neutral" size="small" onClick={() => refetch()}>
        다시 불러오기
      </TextButton>
    </div>
  );
}
if (data.length === 0) return <Feedback type="area" status="empty"
title="가입한 보험이 없어요" />;
```

6.6 오버레이와 알림

`Dialog` 와 `BottomSheet` 는 Radix Dialog 기반이라 `open` · `onOpenChange` 로 제어한다. Esc 와 바깥 누름으로 닫히고, 닫히면 열었던 요소로 포커스가 돌아간다.

TSX

```tsx
// src/features/consult/consult-booking.tsx
'use client';

import { useState, type ReactElement } from 'react';
import { BottomSheet, BottomSheetContent, Button, Dialog, DialogContent,
useToast } from '@nextlab/ds/hanwha';

export function ConsultBooking(): ReactElement {
  const [confirmOpen, setConfirmOpen] = useState(false);
  const [noticeOpen, setNoticeOpen] = useState(false);
  const toast = useToast();

  const book = (): void => {
    setConfirmOpen(false);
    toast.show('상담을 예약했어요', { status: 'success' });
  };

  return (
    <>
      <Button onClick={() => setConfirmOpen(true)}>상담 예약</Button>
      <Button color="neutral" variant="outline" onClick={() =>
setNoticeOpen(true)}>유의사항</Button>

      <Dialog open={confirmOpen} onOpenChange={setConfirmOpen}>
        <DialogContent
          title="상담을 예약할까요?"
          footer={
            <>
              <Button variant="outline" color="neutral" onClick={() =>
setConfirmOpen(false)}>취소</Button>
              <Button onClick={book}>예약하기</Button>
            </>
          }
        >
          예약한 시간에 상담사가 전화를 드려요.
        </DialogContent>
      </Dialog>

      <BottomSheet open={noticeOpen} onOpenChange={setNoticeOpen}>
        <BottomSheetContent title="유의사항" footer={<Button onClick={() =>
```

```
setNoticeOpen(false)}>확인</Button>}>
          가입 전에 상품설명서와 약관을 확인해 주세요.
        </BottomSheetContent>
      </BottomSheet>
    </>
  );
}
```

컴포넌트	규칙
DialogContent	footer 에 Button 1~3개. 확인(Brand Solid)을 맨 오른쪽, 나머지는 variant="outline" color="neutral". footer 의 Button 은 size 를 않으면 Large(48px). 창 폭은 variant 로 정한다: default 최대 552px, fullPopup 최대 744px, 둘 다 화면 폭에서 좌우 여백을 뺀 값을 넘지 않는다
BottomSheetContent	버튼 2개는 footer={<ButtonGroup size="large" secondary={…} primary 목록을 꽉 채우려면 flushBody
useToast().show()	두 번째 인자 { status, action, duration }. status 는 success · warning · error
Tooltip	TooltipProvider 아래에서 Tooltip > TooltipTrigger asChild + TooltipContent 로 쓴다

6.7 내비게이션

TSX

```tsx
// src/app/(main)/main-nav.tsx
'use client';

import Link from 'next/link';
import { usePathname } from 'next/navigation';
import type { ReactElement } from 'react';
import { BottomNavigation, BottomNavigationItem, Icon } from
'@nextlab/ds/hanwha';

const TABS = [
  { href: '/', label: '홈', icon: 'house' },
  { href: '/contracts', label: '내 보험', icon: 'document' },
  { href: '/my', label: '마이', icon: 'profile' },
] as const;

export function MainNav(): ReactElement {
  const pathname = usePathname();
  return (
    <BottomNavigation fixed>
      {TABS.map((tab) => (
        <BottomNavigationItem key={tab.href} asChild label={tab.label}
icon={<Icon name={tab.icon} />} selected={pathname === tab.href}>
          <Link href={tab.href} />
        </BottomNavigationItem>
      ))}
    </BottomNavigation>
  );
}
```

`asChild` 를 주면 자식 요소(여기서는 `Link`)가 실제 요소가 되고, 아이콘과 라벨은 그 안에 그려진다. 선택된 항목에는 `aria-current="page"` 가 붙는다. Header 오른쪽 버튼은 `rightActions={<HeaderIconButton aria-label="알림" onClick={openAlarms}><Icon name="alarm" /></HeaderIconButton>}` 처럼 `HeaderIconButton` 으로 넣는다.

6.8 DS 에 없는 컴포넌트 만들기

먼저 6.3 과 6.4 에서 기존 컴포넌트 조합으로 되는지 확인한다. 직접 만들어야 하면 아래 규칙을 지켜 DS 컴포넌트와 같은 모양과 동작을 맞춘다. 여러 프로젝트에서 같은 요구가 나오면 DS 에 요청한다.

항목	사용
색·글자·간격·모서리·그림자	4장의 토큰 클래스
hover·누름 덮개	`state-layer interaction-overlay`. 비활성(`disabled` · `aria-disabled="true"`) 요소에는 그려지지 않는다
포커스 링	`focus-visible:outline-2 focus-visible:outline-stroke-focus-de`
비활성	`text-fg-disabled-default` · `bg-bg-disabled-default`
클래스 병합	`cn()` (clsx + tailwind-merge)
입력 필드 틀	`FieldArea` 로 감싸면 라벨·도움말·오류 문구가 DS 필드와 같은 자리에 그려진다

TSX

```tsx
// src/features/plan/plan-option.tsx — 규칙을 보여 주는 예시
import type { ReactElement } from 'react';
import { cn, Icon } from '@nextlab/ds/hanwha';

interface PlanOptionProps {
  title: string;
  description: string;
  onSelect: () => void;
  className?: string;
}

export function PlanOption({ title, description, onSelect, className }:
PlanOptionProps): ReactElement {
  return (
    <button
      type="button"
      onClick={onSelect}
      className={cn(
        'state-layer interaction-overlay flex w-full items-center gap-3
rounded-r3 bg-bg-layer-default p-layout-card-padding text-start shadow-
elevation-small',
        'focus-visible:outline-2 focus-visible:outline-stroke-focus-
default',
        className,
      )}
    >
      <span className="flex min-w-0 flex-1 flex-col gap-1">
        <span className="type-body-large-strong text-fg-neutral-default">
{title}</span>
        <span className="type-body-medium text-fg-neutral-muted">
{description}</span>
      </span>
      <Icon name="arrow-right" size="sm" className="text-fg-neutral-
tertiary" />
    </button>
  );
}
```

`FieldArea` 는 `htmlFor` 를 기준으로 도움말에 `{id}-help`, 오류 문구에 `{id}-error` id 를 붙인다. 입력 요소의 `aria-describedby` 와 `aria-invalid` 는 직접 연결한다.

TSX

```
<FieldArea label="계좌 번호" htmlFor="account" helpText="숫자만 입력해 주세요"
errorText={error}>
  <AccountNumberInput
    id="account"
    aria-invalid={error !== undefined}
    aria-describedby={error !== undefined ? 'account-error' : 'account-
help'}
  />
</FieldArea>
```

---

7 폼

폼은 React Hook Form 과 Zod 로 구성한다. 연결 방식은 컴포넌트가 값을 다루는 방식에 따라 두 가지다.

종류	컴포넌트	연결
입력형	`TextField` `PasswordField` `TextArea` `DateField`	ref, `name`, `onChange`, `onBlur` 를 `<input>` 에 전달하므로 `{...register('name')}` 를 그대로 펼친다
선택형	`SelectField` `Segment` `RadioGroup`	`value` · `onValueChange` 를 `Controller` 로 연결한다
체크형	`Checkbox` `Switch`	`checked` · `onCheckedChange` 를 `Controller` 로 연결한다

`errorText` 를 넘기면 필드가 오류 상태가 되고, `aria-invalid` 와 `aria-describedby` 가 자동으로 연결된다. 오류가 있는 동안 `helpText` 는 숨겨진다.

입력값은 화면에 보이는 문자열 그대로 넘어온다. `format="amount"` 는 `"1,000,000"` 처럼 쉼표가 들어가고, `DateField` 는 `"2026.10.30"` 처럼 `YYYY.MM.DD` 형식이다. 숫자나 날짜로 바꾸는 일은 스키마에서 한다.

TS

```
const paymentSchema = z.object({
  amount: z
    .string()
    .transform((value) => Number(value.replaceAll(',', '')))
    .pipe(z.number().min(10_000, '1만 원 이상 입력해 주세요')),
  birthDate: z.string().regex(/^\d{4}\.\d{2}\.\d{2}$/, '생년월일을
YYYY.MM.DD 로 입력해 주세요'),
});
```

TSX

```tsx
// src/features/apply/apply-form.tsx
'use client';

import type { ReactElement } from 'react';
import { Controller, useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { Button, ButtonGroup, SelectField, TextField } from
'@nextlab/ds/hanwha';

const applySchema = z.object({
  name: z.string().min(1, '이름을 입력해 주세요'),
  job: z.string().min(1, '직업을 선택해 주세요'),
});

export type ApplyInput = z.infer<typeof applySchema>;

const JOB_OPTIONS = [
  { value: 'office', label: '사무직' },
  { value: 'field', label: '현장직' },
];

interface ApplyFormProps {
  onSubmit: (input: ApplyInput) => void;
}

export function ApplyForm({ onSubmit }: ApplyFormProps): ReactElement {
  const {
    register,
    control,
    handleSubmit,
    formState: { errors },
  } = useForm<ApplyInput>({ resolver: zodResolver(applySchema) });

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate className="flex
flex-col gap-6">
      <TextField label="이름" {...register('name')} errorText=
{errors.name?.message} />
      <Controller
```

```
          name="job"
          control={control}
          render={({ field }) => (
            <SelectField
              label="직업"
              options={JOB_OPTIONS}
              value={field.value}
              onValueChange={field.onChange}
              errorText={errors.job?.message}
            />
          )}
        />
        <ButtonGroup fixed primary={<Button type="submit">다음</Button>} />
      </form>
    );
}
```

---

8 아이콘·그래픽

아이콘은 `Icon`, 그림은 `Graphic` 에 이름을 넘겨 쓴다. 이름은 타입으로 검사되므로 에디터 자동완성에서 고른다.

8.1 Icon

TSX

```tsx
import { Icon, IconButton } from '@nextlab/ds/hanwha';

<>
  <Icon name="search" />                  {/* md 24px (기본) */}
  <Icon name="arrow-right" size="sm" />   {/* 20px */}
  <Icon name="alarm" size="lg" />         {/* 28px */}
  <Icon name="help" label="도움말" />     {/* 의미를 전달하는 아이콘 */}
  <IconButton aria-label="닫기" icon={<Icon name="close" />} />
</>
```

prop	값	설명
name	IconName	문자열 리터럴 유니언. 없는 이름은 타입 오류다. 전체 목록은 ICON_NAMES
size	"sm" · "md" · "lg" · number	20 · 24 · 28px. 기본 md. 24px 이상은 헤더용 글리프, 미만은 콘텐츠용 글리프를 쓴다
label	string	주면 role="img" 와 aria-label 을, 없으면 aria-hidden 을 붙인다

색은 `currentColor` 다. 부모나 아이콘에 `text-fg-*` 클래스를 주어 바꾼다.

8.2 Graphic

TSX

```
import { Graphic } from '@nextlab/ds/hanwha';

<>
  <Graphic name="illust/umbrella" />                    {/* 그룹 기본 크기
44px */}
  <Graphic name="feedback/empty" size={80} />
  <Graphic name="stock/kb" size={40} label="KB증권" />
</>
```

`name` 은 `그룹/이름` 형식의 `GraphicName` 타입이다. `size` 는 표시 폭(px)이고 높이는 원본 비율을 따른다. 그래픽은 고정색 SVG 라 색을 바꿀 수 없고 다크 모드에서도 같은 색으로 표시된다.

그룹	개수	기본 크기	용도
illust/	145	44px	카테고리·기능 강조
feedback/	17	64 · 80px	빈 상태, 오류, 완료, 인증 상태
avatar/	20	72 · 56px	사람 아바타(연령·성별, 상담원·설계사)
logo/	3	원본 크기	한화 로고(PC·태블릿·스플래시)
stock/	24	40px	증권사 마크
cert/	4	44px	간편인증 사업자 마크

`content/` (본문 일러스트, 450개)와 `banner/` (배너 일러스트, 450개)는 AI 생성 이미지다. 이 두 그룹만 이름별 조각으로 나뉘어 쓸 때 받고, 나머지 그룹은 한 묶음으로 들어간다. 번들 영향은 10.3 에서 다룬다.

9 접근성

키보드 조작, 포커스, 낭독 연결은 컴포넌트가 처리한다. 화면 코드에서는 9.2 의 항목만 챙긴다.

9.1 컴포넌트가 처리하는 범위

항목	동작
키보드·포커스	Radix 기반 컴포넌트는 방향키 이동, Esc 닫기, 오버레이 포커스 가두기와 복귀를 처리한다
펼침 목록	PC 의 `SelectField` 펼침 목록은 WAI-ARIA select-only combobox 패턴이다. 포커스는 칸에 두고 `aria-activedescendant` 로 줄을 가리킨다. ↑↓ · Home · End · PageUp · PageDown 이동, Enter · Space 선택, Alt+↑ 선택 후 닫기, 글자 입력으로 해당 글자로 시작하는 줄 이동을 지원한다
필드 연결	`label` 과 입력 요소를 `id` 로 연결하고, `helpText` · `errorText` 를 `aria-describedby` 로, 오류 상태를 `aria-invalid` 로 노출한다
다이얼로그	`description` 이 없으면 제목을 화면낭독기용 설명으로 넣는다. 제목이 없는 `AlertDialog` 는 본문을 창 이름으로 쓴다
상태 알림	`Feedback status="error"` 는 `role="alert"` 로 낭독되고, `SkeletonGroup` 은 `label`(기본 "불러오는 중")을 낭독한다
금액 증감	`AmountChange` 는 ▲▼ 기호를 낭독하지 않는다. 방향은 글자(예: "1,200원 올랐어요")로 전한다
아이콘·그래픽	`label` 이 있으면 `role="img"`, 없으면 `aria-hidden` 으로 처리한다
글자 확대	`textScale="large"` 에서 `type-*` 텍스트가 1.125배로 커진다. Header 높이·아이콘은 고정되고 제목 글자만 커진다
동작 줄이기	`prefers-reduced-motion` 에서 Skeleton·Spinner·Carousel 애니메이션을 멈추거나 줄인다

9.2 화면 코드의 책임

항목	기준
아이콘 버튼	`IconButton` 에 `aria-label` 필수
페이지 제목	화면당 `h1` 하나. `Header` 제목이 `h1` 이므로 본문에 제목이 따로 있으면 `titleAs="p"`
색	색만으로 의미를 구분하지 않는다. 오류는 문구나 아이콘을 함께 표시한다
대비	본문 텍스트 4.5:1 이상. 본문 글자색은 9.3 의 토큰에서 고른다
터치 영역	48px 이상
그래픽	의미를 전달하면 `label`, 장식이면 생략
문서 언어	`<html lang="ko">`

9.3 본문 글자색

본문 크기 글자에는 아래 토큰을 쓴다. 페이지 바탕(`bg-layer-base`) 위에서 WCAG 2 상대 휘도 공식으로 잰 대비이며, 모두 본문 기준 4.5:1 을 넘는다.

글자 토큰	light 값	light 대비	dark 대비
`fg-neutral-default`	`#272B2F`	14.26	14.85
`fg-neutral-strong`	`#403F3E`	10.51	14.85
`fg-neutral-muted`	`#5E5C5A`	6.66	13.33
`fg-brand-strong`	`#9E3F00`	6.64	9.42
`fg-warning-default`	`#774408`	8.03	12.64
`fg-trend-up`	`#D22E3C`	5.00	6.48

글자 토큰	light 값	light 대비	dark 대비
`fg-trend-down`	`#415CAC`	6.27	8.64

회색 면(`bg-neutral-tint`) 위의 `fg-neutral-muted` 는 light 6.11, dark 10.59 다. 그 밖의 글자 토큰(`subtle` `tertiary` `brand-default` 와 상태 색)은 큰 글자, 같은 뜻의 글자가 옆에 있는 아이콘·보조 표시, 장식에 쓴다.

9.4 자동 검사

DS 저장소는 Storybook 스토리를 실제 Chromium 에서 렌더링한 뒤 axe 로 검사하고, 위반이 있으면 시험을 실패시킨다.

화면 코드도 같은 방식으로 검사할 수 있다. jsdom 은 레이아웃과 색을 계산하지 않으므로, 대비까지 보려면 Playwright 같은 실제 브라우저에서 axe 를 돌린다.

---

10 성능

DS 를 넣었을 때 앱이 받는 CSS·JS 크기와, 그 크기가 나오는 이유다. 0.9.0 배포물을 esbuild 0.28 로 번들해(코드 분할, minify, React 제외) 쟀고, 압축은 Node zlib 기본 설정이다. 1KB 는 1,024바이트다.

10.1 CSS

단계	크기
`styles.css` 원본	1,034KB
앱 빌드에서 minify 한 뒤	884KB
전송 (gzip)	70KB

단계	크기
전송 (brotli)	38KB

`styles.css` 는 쓰는 컴포넌트와 무관하게 한 파일을 통째로 받는다. 원본의 24%(251KB)는 범위 선택자 `[data-nxl-surface="hanwha-customer"]` 가 6,945번 반복된 것이고, 같은 문자열의 반복이라 압축하면 대부분 사라진다. 클래스는 약 6,800개, 토큰 변수는 504개다.

10.2 JS 초기 번들

컴포넌트 하나만 import 했을 때 초기 번들(gzip)이다. 초기 번들은 진입 모듈이 정적으로 불러오는 조각의 합이며, 동적으로 불러오는 차트 엔진과 큰 그림은 넣지 않았다.

import	gzip
`HanwhaProvider`	1KB
`Skeleton` · `Badge` · `Table` · `PageContainer`	10KB
`Button`	13KB
`Tabs` · `RadioGroup`	20KB
`Icon`	27KB
`TextField`	31KB
`Checkbox`	34KB
`Dialog` · `BottomSheet`	44KB
`DateField`	52KB
`Accordion`	60KB
`Graphic` · `Feedback` · `Avatar`	236–237KB

import	gzip
`List` · `ListItem`	240–241KB
`Header`	262KB
`SelectField`	271KB
`Button` `TextField` `Checkbox` `Dialog` `Header` 함께	272KB

컴포넌트는 모듈별 ESM 파일이고 `sideEffects` 가 CSS 로 한정되어 있어, 쓰지 않는 컴포넌트는 번들에서 빠진다. 230KB 안팎으로 뛰는 줄은 모두 10.3 의 정적 그래픽 묶음을 포함한다. 묶음은 여러 컴포넌트가 함께 써도 한 번만 들어가므로 마지막 줄이 272KB 에 그친다.

10.3 정적 그래픽 묶음

`Graphic` 은 이름 문자열을 받아 실행 중에 그림을 찾는다. 번들러는 어떤 이름이 쓰일지 알 수 없으므로 정적 그룹(`illust` `feedback` `avatar` `logo` `stock` `cert` , 213개)을 한 모듈에 묶어 통째로 넣는다. 크기는 gzip 약 230KB 다. `content` `banner` (900개)는 이름별 조각으로 나뉘어 화면에서 쓸 때만 받는다.

이 묶음을 끌어오는 컴포넌트는 `Graphic` `Feedback` `Avatar` `List` `ListItem` `Loading` `Header` `SelectField` 와 차트다. `Header` 와 `List` 는 대부분의 화면이 쓰므로 이 묶음은 앱의 기본 비용으로 잡는다. Next.js 에서 `Graphic` 을 Server Component 에서만 쓰면 그림은 HTML 로만 가지만, 위 목록의 클라이언트 컴포넌트를 하나라도 쓰면 묶음 전체가 클라이언트 번들에 들어간다.

큰 그림은 받는 동안 같은 크기의 빈 자리를 그린다. `Graphic` 은 큰 그림일 때만 자기 안에 `Suspense` 경계를 두는데, 경계를 둘지는 받았는지가 아니라 이름으로만 정한다. 서버(이미 받음)와 브라우저(처음 받음)의 트리 구조가 같아야 하이드레이션이 어긋나지 않기 때문이다.

10.4 폰트

한화고딕 woff2 는 굵기마다 약 1.2~1.3MB 이고 글자 일부만 담은 subset 판이 없다. 첫 화면에는 본문(300)과 제목(700) 두 굵기가 필요하다. `font-display: swap` 이라 글자는 시스템 글꼴로 먼저 그려지고 폰트를 받은 뒤 바뀌며, 이때 글자 폭이 달라져 줄바꿈이 움직일 수 있다.

11 품질 보증

DS 저장소가 갖춘 검사와 0.9.0 의 결과다. 화면 코드가 DS 에 맡겨도 되는 것과 직접 확인해야 하는 것을 가르는 기준이다.

검사	내용
단위 시험	Vitest · Testing Library · jsdom 으로 props 동작, 키보드 조작, ARIA 연결, Figma 수치(높이·간격)를 확인한다
접근성	모든 Storybook 스토리를 Chromium 에서 렌더링하고 axe 로 검사한다. 위반이 있으면 실패한다 (9.4)
토큰 생성물	생성기의 `--check` 모드. Figma Variables 에서 만든 결과와 커밋된 CSS 가 다르면 실패한다
패키지 경계	패키지 사이의 순환 의존, 내부 경로 import, 선언하지 않은 의존을 막는다
설치 확인	tgz 를 저장소 밖의 새 Vite 프로젝트(pnpm · Tailwind v4 · React 19)에 설치하고 import, Provider, 프로덕션 빌드를 확인한다
화면 비교	Storybook 을 Chromatic 에 게시해 변경 전후 화면을 비교한다
문서 예시	이 문서의 예시 코드를 배포 타입 선언에 strict 모드로 컴파일하고 `@deprecated` 사용을 찾는다

2026-11-02 에 0.9.0 으로 다시 돌린 결과는 단위 시험 83개 파일 646개 통과, 문서 예시 컴파일 오류 0건, `@deprecated` 사용 0건이다.

화면 코드에서 직접 확인할 것은 세 가지다. 색 대비(9.3), 화면 단위의 접근성(제목 구조, 초점 순서, 대체 텍스트), 3.6 의 클래스 조합이다.

12 설계 결정과 비용

구조를 정한 주요 결정과, 그 결정 때문에 생기는 제약이다. 앞 장의 규칙 대부분이 여기서 나온다.

결정	이유	비용
스타일을 미리 컴파일한 CSS 한 파일로 배포한다. Tailwind 설정이나 CSS-in-JS 로 배포하지 않는다	앱의 Tailwind 버전·설정과 무관하게 같은 결과가 나온다. 실행 중 스타일 주입이 없어 Server Component 와 스트리밍 렌더링에 제약이 없다. 앱이 토큰을 Tailwind 에 등록하지 않아도 된다	쓰지 않는 컴포넌트의 CSS 도 받는다 (10.1). 앱 Tailwind 는 DS 토큰 이름을 몰라 토큰 클래스에 변형 접두사를 붙일 수 없다 (3.5)
모든 규칙을 범위 선택자 아래에 둔다	기존 앱 CSS 와 한 문서에서 공존하고, 화면 일부에만 적용할 수 있다	Portal 은 `syncRoot` 가 있어야 범위에 든다 (3.3). 선택자 반복으로 원본 CSS 가 커진다 (10.1)
리셋만 레이어에 넣고 나머지는 레이어 밖에 둔다	앱 CSS 가 무엇이든 컴포넌트 모양이 바뀌지 않는다. 리셋은 `@layer base` 로 내려 앱 유틸리티를 덮지 않게 했다	앱의 변형 클래스가 DS 의 같은 속성 클래스에 진다 (3.6)
상호작용은 Radix primitives 위에 만든다	키보드·포커스·ARIA 동작을 검증된 구현에 맡기고, 스타일이 없는 primitives 라 Figma 모양을 그대로 입힐 수 있다	DOM 구조와 Portal 위치가 Radix 규칙을 따른다
토큰은 Figma Variables 에서 생성한다	디자이너의 값 변경이 손을 거치지 않고 코드로 넘어오고, 손으로 고친 값은 `--check` 에 걸린다	앱에서 토큰 값을 덮어쓰는 테마 주입을 지원하지 않는다

결정	이유	비용
반응형은 토큰이 맡는다	글자 크기·간격이 Figma 의 Mobile·Tablet·PC 모드와 같은 분기점에서 바뀌어, 화면 코드에 분기 없이 세 폭이 맞는다	토큰은 화면 폭을 따르므로 좁은 영역에 넣어도 글자는 PC 크기다. 영역 폭이 중요한 `PageColumns` 와 PC `Header` 만 컨테이너 쿼리를 쓴다
아이콘·그래픽은 이름으로 받는다	이름이 문자열 리터럴 유니언이라 오타가 타입 오류가 되고, 에디터 자동완성으로 고른다	번들러가 쓰는 그림만 골라낼 수 없어 정적 그래픽 묶음이 통째로 들어간다 (10.3)
영역이 정해진 컴포넌트는 영역 props 로 받는다	영역 순서·간격과 낭독 연결 (`aria-describedby` 등)을 컴포넌트가 보장한다	영역 안 배치를 바꾸려면 컴포넌트를 고쳐야 한다
`'use client'` 를 빌드에서 자동으로 붙인다	상태가 없는 컴포넌트는 Server Component 로 남고, 상태가 있는 컴포넌트도 Server Component 에서 바로 import 할 수 있다	함수 props(`onClick` 등)는 Server Component 에서 넘길 수 없다 (2.5)

13 AI 도구 연동

Storybook 은 MCP(Model Context Protocol) 서버를 함께 제공한다. 등록하면 AI 코딩 도구가 컴포넌트 문서(props, 사용 예, 스토리)를 조회해 코드를 생성한다. 인증은 필요 없다.

도구	동작
`docs-list`	컴포넌트·문서 목록과 ID
`docs-show`	컴포넌트의 props, 사용 예, 스토리

도구	동작
docs-show-story	특정 스토리의 상세

13.1 등록

Shell

```
# Claude Code
claude mcp add --transport http nxl-ds https://main-
--6a794efb546779fb68f18bd0.chromatic.com/mcp
```

Claude Code 는 프로젝트 루트의 `.mcp.json`, Cursor 는 `.cursor/mcp.json` 에 넣는다.

JSON

```
{
  "mcpServers": {
    "nxl-ds": {
      "type": "http",
      "url": "https://main--6a794efb546779fb68f18bd0.chromatic.com/mcp"
    }
  }
}
```

VS Code(GitHub Copilot)는 `.vscode/mcp.json` 에 넣는다.

JSON

```
{
  "servers": {
    "nxl-ds": {
      "type": "http",
      "url": "https://main--6a794efb546779fb68f18bd0.chromatic.com/mcp"
    }
  }
}
```

설정 파일을 저장소에 커밋하면 팀원이 같은 설정을 쓴다.

13.2 규칙 파일

AI 도구의 프로젝트 규칙 파일(`CLAUDE.md`, `.cursorrules`, `.github/copilot-instructions.md`)에 아래 내용을 넣는다.

Markdown

```
## UI 규칙 (NXL-DS Hanwha)
- UI 는 @nextlab/ds/hanwha 컴포넌트로 만든다. 필요한 컴포넌트가 없으면 새로 만들지 말고 알린다.
- props 는 nxl-ds MCP(docs-list → docs-show)로 확인한 것만 쓴다. 에디터에서 취소선이 그어지는 @deprecated 이름은 쓰지 않는다.
- 색은 bg-bg-* · text-fg-* · border-stroke-*, 텍스트는 type-* 만 쓴다. hex 값, text-sm, font-bold 를 쓰지 않는다.
- 본문 크기 글자색은 text-fg-neutral-default · strong · muted, text-fg-brand-strong, text-fg-warning-default 중에서 고른다.
- className 은 배치(margin, width)에만 쓴다. 인라인 style 을 쓰지 않는다.
- 화면 폭 분기는 min-[768px]: · min-[1200px]: 를 쓰고, 기본값 클래스 없이 구간마다 쓴다(max-[767px]:flex min-[768px]:hidden).
- Next.js 에서 이벤트 핸들러·상태가 있는 컴포넌트는 'use client' 파일에 작성한다.
- 하단 고정 버튼은 ButtonGroup fixed, 화면당 Primary 버튼은 하나다.
- 비동기 화면은 Skeleton(로딩) · Feedback status="empty"(빈 상태) · Feedback status="error"(오류)를 모두 구현한다.
- IconButton 에는 aria-label 을 준다.
```

---

14 알려진 제약 (0.9.0)

0.9.0 을 쓸 때 미리 알아야 하는 제약과 대응 방법이다.

항목	내용	대응
폰트	패키지에 폰트 파일이 없고, 한화 폰트 CDN 은 CORS 헤더를 보내지 않는다	CDN 파일을 `public/fonts/` 에 둔다 (2.3)
앱 변형 클래스	Provider 안에서 DS CSS 에 들어 있는 클래스와 같은 속성을 바꾸는 앱의 변형 클래스 (`flex min-[768px]:hidden`)가 무시된다	구간마다 범위 변형으로 쓴다 (3.6)
정적 그래픽 묶음	`Header` `List` 등 대부분의 화면이 쓰는 컴포넌트가 그래픽 묶음 약 230KB(gzip)를 초기 번들에 넣는다	앱의 기본 비용으로 잡는다 (10.3)
화면 안 안내 상자	전용 컴포넌트가 없다	DS 담당과 구성을 확인한다
확장 컴포넌트	`PageContainer` `PageTitle` `PageColumns` `SiteFooter` `SiteMenu` 는 디자이너 검토 중이다	props 가 1.0.0 전에 바뀔 수 있다. 바뀌면 CHANGELOG 에 수정 방법을 적는다
확정 전 항목	`InstitutionPicker` 는 디자이너 정의 전이고 은행 마크 자산이 없다. 상태 아이콘 `info` `warning` `error` 는 디자이너 확정 전이다	서비스 적용 전에 DS 담당과 확인한다
다크 모드	다크 토큰은 있으나 디자이너 검토 중이다	서비스 적용 전에 DS 담당과 확인한다
테마	앱에서 토큰 값을 덮어쓰는 테마 주입은 지원하지 않는다	값 변경은 DS 에 요청한다
폰트 용량	한화고딕 woff2 가 굵기마다 약 1.2~1.3MB 이고 글자 subset 판이 없다	10.4
토큰 클래스 변형	토큰 클래스에 `hover:` 같은 변형 접두사를 붙이면 생성되지 않는다	CSS 변수 문법을 쓴다 (3.5)
모듈 형식	ESM 전용이라 CommonJS `require()`, 기본 설정의 Jest 로 불러올 수 없다	Vitest 를 쓴다 (2.6)

항목	내용	대응
브라우저	iOS Safari 16.4, Chrome 111 미만은 지원하지 않는다	1.3
Tailwind CSS v3	검증하지 않았다	v4 를 쓴다

---

15 버전 정책과 지원

버전 규칙, 새 버전으로 올리는 방법, 버그를 제보하는 방법이다.

15.1 버전 규칙

0.9.0 은 1.0.0 직전 버전이며, 1.0.0 에서는 화면 패턴이 추가된다. 0.9.x 에서도 컴포넌트 이름이나 props 가 바뀔 수 있고, 바뀌면 이전 이름을 `@deprecated` 별칭으로 남긴다. 1.0.0 부터는 Semantic Versioning 을 따른다.

구분	예	변경 범위	업데이트 시 작업
Patch	1.0.1	버그 수정, 시각 보정	없음
Minor	1.1.0	컴포넌트·props·토큰·아이콘 추가 (하위 호환)	없음
Major	2.0.0	export · props · 토큰의 제거나 이름 변경, `@deprecated` 별칭 제거	CHANGELOG 의 수정 방법에 따라 수정

15.2 업데이트

```Shell
pnpm add ./vendor/nextlab-ds-<버전>.tgz
```

- 잠금 파일을 함께 커밋한다.
- 한 앱에는 DS 한 버전만 둔다. 두 버전의 `styles.css` 는 같은 범위 선택자를 쓰므로 서로 덮는다.
- 변경 내용은 배포본과 함께 전달하는 CHANGELOG 와 Storybook 에서 확인한다.
- 토큰 클래스와 props 로만 스타일을 지정한 화면은 업데이트만으로 디자인 변경이 반영된다.

15.3 버그 제보

항목	내용
버전	`@nextlab/ds` 버전 (`package.json` 의 값)
재현	컴포넌트 이름과 최소 재현 코드
환경	화면 폭(px), 브라우저와 버전, `textScale` · `colorMode` 값
결과	기대 결과와 실제 결과(캡처)

화면 폭에 따라 바뀌는 컴포넌트가 많으므로 화면 폭은 빠뜨리지 않는다.

---

16 문제 해결

자주 만나는 증상과 해결 방법이다. 오류 문구나 증상으로 찾는다.

증상	원인
TypeScript 에서 `Cannot find module '@nextlab/ds/hanwha'`	`moduleResolution` 이 `node` 다
Jest 에서 `Cannot use import statement outside a module`	패키지가 ESM 전용이
테스트에서 `ResizeObserver is not defined` · `matchMedia is not a function`	jsdom 에 없는 브라우 API 다
iOS 에서 하단 고정 버튼·하단 메뉴가 홈 인디케이터와 겹친다	뷰포트에 `viewport-fit=cov` 가 없다
`hover:bg-bg-…` 같은 클래스가 적용되지 않는다	토큰 클래스와 변형 접두사 조합은 생성되지 않는다
앱의 고정 요소가 다이얼로그 위로 올라온다	z-index 가 100 이상이다
컴포넌트가 스타일 없이 표시된다	`styles.css` 를 impo 하지 않았거나 요소가 `HanwhaProvider` 밖 있다
다이얼로그·바텀시트·툴팁·펼침 목록만 스타일이 없다	Portal 내용이 범위 선택자 밖에 있다
Next.js 에서 `Event handlers cannot be passed to Client Component props`	Server Component 에서 `onClick` 등 함수 넘겼다
`useToast` 또는 `Tooltip` 에서 런타임 오류	`ToastProvider` · `TooltipProvider` 없다
메뉴·푸터 링크를 누르면 페이지를 통째로 다시 받는다	`linkComponent` 를 넘기지 않았다

증상	원인
한화고딕이 적용되지 않는다	`/fonts/` 에 파일이 없거나, 하위 경로(basePath) 배포로 `/fonts/` 가 404 이거나, CDN CSS 를 직접 참조했다
PC 에서 글자 크기가 바뀌지 않거나 큰 글자 모드가 적용되지 않는다	`text-sm` · `leading-` 등 Tailwind 기본 텍스 클래스를 썼다
1024–1199px 에서 레이아웃이 어긋난다	Tailwind `lg:`(1024p 로 분기했다
Provider 안에서만 `min-[768px]:hidden` 같은 반응형 클래스가 적용되지 않는다	같은 요소의 기본값 클래스(`flex` 등)가 DS CSS 의 레이어 밖 규칙이라 이긴다
`syncRoot` 를 켠 뒤 Provider 밖 영역의 여백·글꼴이 바뀌었다	`<html>` 이 범위가 되어 DS 리셋이 문서 전체에 적용된다
`dialogFromTablet` 에서 타입 오류가 난다	`BottomSheet` 에 넘겼다
`Header` 하나를 넣었는데 초기 번들이 230KB 늘었다	정적 그래픽 묶음이 함께 들어온다
토큰 클래스(`bg-bg-*` 등)가 적용되지 않는다	요소가 Provider 범위 밖에 있다
에디터에서 컴포넌트 이름에 취소선이 그어진다	이전 화면 호환용으로 남긴 이름이다

NXL-DS Hanwha 0.9.0 개발자 가이드 · 2026-11-02